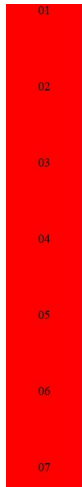


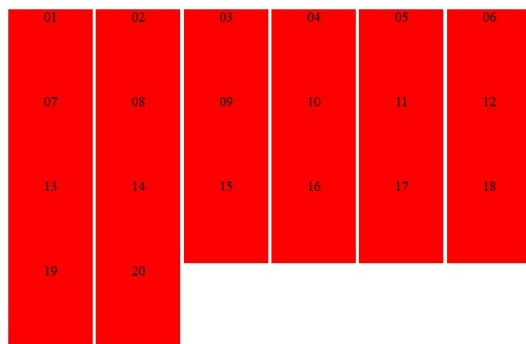
Opdracht 1



Bij block staan ze onder elkaar.



Bij inline zijn het kleine blokjes.



Bij inline-block zijn het grote blokken.

Opdracht 2

```
<body>  
<div class="area">01</div><div class="area">02</div>
```



Wanneer de div op een nieuwe lijn begint is er een spatie tussen de blokken.

Opdracht 3

De tekst staat horizontaal gecentreerd omdat dat zo in de css staat. "div{text-align: center;}"

Vertical-align doet simpelweg niets op een normaal block-element zoals een div. Het werkt alleen in een paar specifieke situaties: in tabelcellen (`display: table-cell`), tussen inline-block elementen onderling, of bij inline elementen ten opzichte van de lijnbox. Een standaard div valt niet in die categorie, dus `vertical-align: center` heeft daar geen enkel effect.

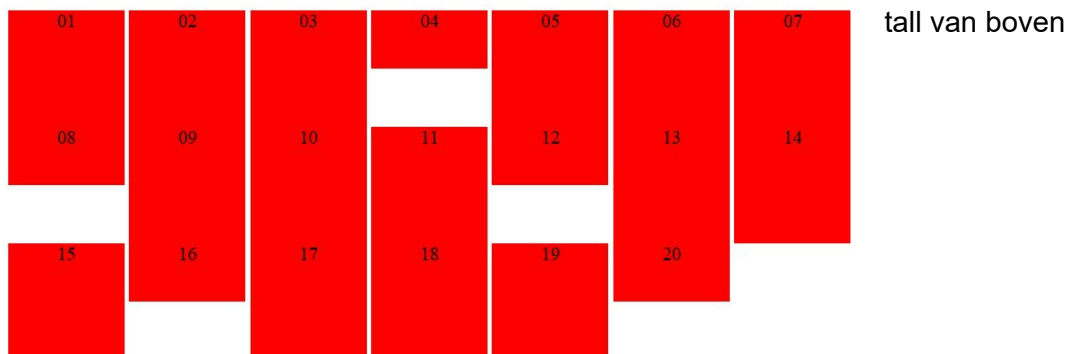
`vertical-align: middle` is nooit gemaakt om iets netjes in het verticale midden van een div te zetten. Het heeft daar werkelijk niks mee te maken. Het zegt alleen maar: “zet dit inline (of inline-block) element in het midden van de lijnbox vergeleken met andere inline dingen ernaast.” Dus het vergelijkt elementen binnen dezelfde tekstregel, niet de totale hoogte van een container. Een div is gewoon een block-element met een eigen rechthoek, en heeft helemaal geen “lijnbox” waar `vertical-align` iets mee kan. Dus je kunt roeptoeteren met `vertical-align: middle` wat je wil, maar een div haalt zijn schouders op en gaat verder met zijn dag.

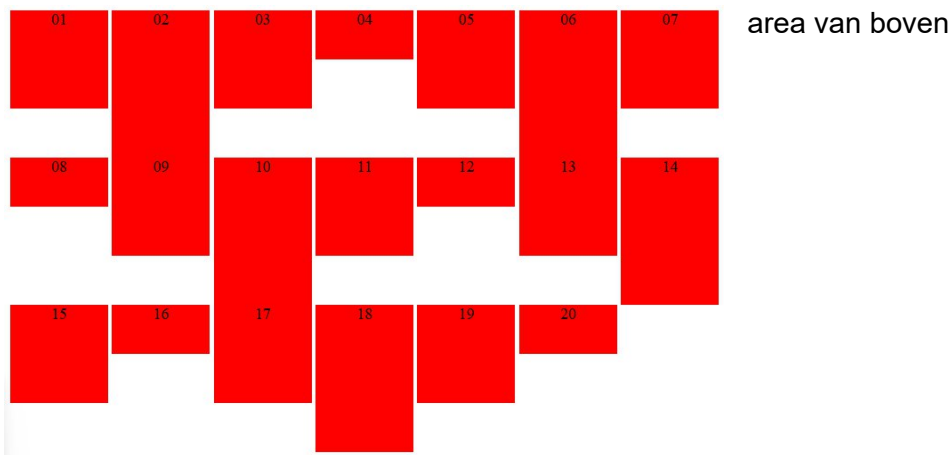
De simpelste manier om één regel tekst verticaal te centreren is met `line-height == height`.

Bv, `.area{height: 100px; line-height:100px}`

Zodra je een element `display: flex` geeft en `align-items` op `center` zet, worden de kinderen automatisch verticaal gecentreerd. Combineer je dat met `justify-content: center`, dan zit alles zowel horizontaal als verticaal netjes in het midden. Dit is tegenwoordig de standaardmanier, simpel en betrouwbaar.

Sinds `tall` en `area` even specifiek zijn, wint `tall` omdat het na `area` staat.





Opdracht 4

Als je een element `float:left` geeft, stapt het uit de normale stroom, schuift zo ver mogelijk naar links en alle andere niet-gefloatte elementen laten hun positie eromheen aanpassen.

Er blijft geen gat over omdat 09 als float uit de normale stroom verdwijnt, waardoor 08 en 10 elkaar direct zien en vanzelf tegen elkaar aanschuiven.

Dat floats zich soms wél naast elkaar zetten en soms niet, komt simpelweg doordat ze elkaar nog steeds in documentvolgorde volgen en elk voor zich proberen zoveel mogelijk naar links te schuiven. Als er genoeg horizontale ruimte is, kunnen 12 en 14 gewoon naast elkaar liggen. Maar zodra de beschikbare breedte te klein wordt, past 14 niet meer naast 12 en zakt hij automatisch naar de volgende “regel”. Floats maken zelf geen nette rijtjes; ze passen zich alleen aan de beschikbare ruimte aan.

Waarom is er geen kleine ruimte tussen floaty (groen) en rood ernaast?

Omdat een float zich niet gedraagt als een inline-block. Die kleine witruimte die je normaal ziet tussen twee inline-block elementen komt door de spaties/line breaks in de HTML. Maar zodra één van de twee een float wordt, valt dat gedrag weg: een float is een block-achtige box en whitespace tussen een float en een inline-level element wordt simpelweg genegeerd. Daarom zie je tussen twee inline-blocks wel zo'n klein “HTML-spatie-randje”, maar tussen een float en een inline-block helemaal niks — die schuiven strak tegen elkaar aan.

`clear: both` werkt alleen voor elementen die echt als blok in de normale block-flow meedoen. Een inline-block hoort daar simpelweg niet bij.

In jouw geval is 17 wel blauw (de CSS pakt), maar het element kan niet “clearen” omdat inline-blocks niet reageren op `clear`; ze zweven gewoon verder alsof er niets aan de hand is. Maak het element even `display:block` en je ziet direct dat het wél onder alle floats duikt. Daarmee wordt duidelijk dat het probleem niet de `clear` is, maar het inline-block gedrag.

Opdracht 5

Bij het groter maken van het venster ontstaan meerdere rijen floats rond 09 omdat alle .area-elementen float:left hebben. Als je element 13 de break-class geeft met clear: both, wordt 13 gedwongen onder alle voorgaande floats te beginnen en komt het dus onder 09 te staan. clear is vooral nuttig bij lay-out met zwevende blokken; bij een simpel inline stukje tekst dat rond een float loopt, pas je clear in de praktijk niet toe.